

About

- **Author: Alessio Giorgetti, Associate Professor, Department of Information Engineering (DII), University of Pisa, Italy.**
- Contributor: ASM Chafiullah, Researcher Assistant, DII, University of Pisa.

This tutorial explains how we can create a standalone image featuring **Kathara** and **ONOS**. The image can also be obtained from **dockerhub** using the following link:

Required Files

First clone this repository from github:

```
git clone https://github.com/bhuiyan-chafi/sdn-docker.git
```

Then navigate to the desired directory:

```
# from where you have cloned the repo
cd sdn-docker/kathara/katharaXonos/
```

Create from Scratch

Run the following command to build the image from scratch:

```
# From the directory: katharaXonos/
docker build -t kathara/onos-classic . --no-cache
```

Once the image is built, test the following scenarios to ensure everything is working correctly:

1. Creating a container and connecting to its shell:

- Make sure to expose the necessary ports for the ONOS Web UI (**8181**) and SSH (**8101**):

```
docker run -it --rm -p 8181:8181 -p 8101:8101 --name kathara-
onos-test kathara/onos-classic /bin/bash
```

- However, in our setup, we have configured **ssh** connectivity natively within the container, and we will connect to the **ONOS CLI** from there. So, use this command to run the container for our specific use case:

```
docker run -it --rm -p 8181:8181 --name kathara-onos-test
kathara/onos-classic /bin/bash
```

- Test external connectivity from inside the container:

```
ping google.com
```

2. Verifying Kathará base networking tools:

- From inside the container shell, verify that the standard networking and system tools inherited from the `kathara/core` base image are available:

```
# Check networking utilities
ip a
iptables --version
tcpdump --version
ping -V
```

3. Verifying Python, NetworkX, and Java environments:

```
# Verify Java (should be OpenJDK / Temurin 11)
java -version

# Verify Python 2 (required for Bazel ONOS toolchains)
python2 -V

# Verify Python 3 and NetworkX
python3 -V
python3 -c "import networkx as nx; print(f'NetworkX version:
{nx.__version__}')"

```

4. Verifying Bazel & ONOS installation:

```
# Verify Bazel version
bazel version

# Verify ONOS source folder
ls -la $ONOS_ROOT
```

5. Starting and testing ONOS:

- Run the preconfigured helper script to start ONOS:

```
start-onos
```

- Once ONOS has booted up, test accessing the ONOS CLI:

```
source $ONOS_ROOT/tools/dev/bash_profile
onos onos@localhost
```

*(Default credentials: user **onos**, password **rocks**)*

6. Web UI

- **ONOS** is exposed to our localhost's 8181 port. Navigate to <http://localhost:8181/onos/ui> in your web browser to access the GUI. Use **onos** as the username and **rocks** as the password to log in.
- If you have completed all these steps, you now have a working version of an ONOS node built directly on top of the **kathara/core** image. By using this custom image, we can spawn a **controller** node within Kathará that completely bypasses the host machine's **iptables** firewall. This works because the controller is spawned natively as a Kathará node and its traffic is strictly managed within an isolated Kathará collision domain.
- The following tutorial will explain how to set up a linear topology with 2 **ovs-switches**, 2 hosts, and an SDN Controller (ONOS) using Kathará.

If you want to simulate a LAB

1. Setting up Kathará:

- Set up **Kathará** by following these [steps](#).

2. Setting up the Lab:

- Execute the [startKatharaXonos.sh](#) script:

```
# If the image is missing and needs to be built
bash startKatharaXonos.sh build

# If the image is already built
bash startKatharaXonos.sh run
```